Homework 1: 21D012 Machine Learning for Finance

Marco Canal, Michael Duarte Gonçalves and Olivia Peter

15 May 2026

Abstract

This homework asks where financial predictability is real enough to survive formal testing and out-of-sample evaluation. We study that question in three settings. First, we estimate Granger networks for ten liquid assets to separate return lead-lag effects from volatility spillovers. Second, we use neural networks, LSTMs and Gaussian processes to forecast S&P 500 returns, log prices and directions from the assigned Goyal-Welch predictors. Third, we compare the machine-learning forecasts with ARMA-GARCH baselines. The evidence is asymmetric. Weekly return causality is sparse. The denser monthly return network is less secure because the sample is short. EWMA volatility shows clearer propagation across equities, commodities, crypto and Treasuries. Gaussian processes fit log prices well because the target inherits the previous level, yet return gains over the mean benchmark are economically negligible. Direction accuracy is fragile once MCC is considered. The main lesson is that volatility transmission is more credible than exploitable one-month return prediction in this sample.

Contents

1 Asset Networks 3

1a Data, EWMA Volatility and Granger Setup 3

1b Return Causality Networks 6

1c Volatility Spillover Networks 8

2 S&P 500 Forecasting 11

2a Feature Selection and Target Design 12

2b Return Forecasts 13

2c Log-Price Forecasts 14

2d Direction Forecasts 16

2e Sensitivity and Target Comparison 17

3 ARMA-GARCH Benchmarks 18

3a Model Orders and Volatility Persistence 19

3b ML Versus ARMA-GARCH 21

4 Summary of Lessons 23

5 Contributions 25

References 25

1 Asset Networks

The first exercise studies two directed systems for SPY, XLK, XLF, XLE, GLD, SLV, GSG, BTC-USD, ETH-USD and TLT over the assigned 2024 to 2025 period: one system for return predictability and one system for volatility spillovers. The data step retrieves daily adjusted closes from Yahoo Finance, aligns ETF and crypto observations to the same trading calendar and forward-fills at most 1 day when a sporadic crypto gap would otherwise break the panel. Granger regressions compare $X_{t-\ell}$ with Y_t at the same calendar index. If one series carries a stale close while the other series uses a fresh close, the lagged regressor can describe the calendar mismatch rather than a market response. Aligning the panel first keeps that error out of the test. The notation follows the return material from Lecture 2 and the HW1 handout. Let P_t be the adjusted price at date t, let R_t be the corresponding simple one-period return and let r_t be the log return:

$$r_t = \ln(1 + R_t) = \ln\left(\frac{P_t}{P_{t-1}}\right) = \ln P_t - \ln P_{t-1}, \qquad r_t[k] = r_t + r_{t-1} + \cdots + r_{t-k+1}.$$

The second expression, $r_t[k]$, is the k-period log return. It tells us how returns aggregate over time: a multi-period log return is obtained by adding one-period log returns instead of multiplying gross returns. The additive identity is what lets the later AR, VAR and GARCH equations operate directly on returns with linear lag terms. The notation also gives a first expectation before we look at the tables. Return levels should be noisy because they measure new information arriving between two dates. Volatility should be more persistent because it is built from squared returns. The squaring operation keeps the size of shocks even after their sign has disappeared.

1a Data, EWMA Volatility and Granger Setup

The daily adjusted-close panel contains 502 cleaned observations. Weekly Friday sampling gives 104 weekly returns. Month-end sampling gives 23 monthly returns. Volatility follows the EWMA definition used in Lecture 2 and Lab session 1:

$$\sigma_t^2 = \lambda\sigma_{t-1}^2 + (1-\lambda)r_{t-1}^2, \qquad \lambda = 0.94.$$

Here σ_t^2 is the variance estimate carried into date t, σ_{t-1}^2 is yesterday's estimate and r_{t-1}^2 is the most recent squared log return. The update is recursive. Each date keeps the fraction λ of the old variance estimate and adds the fraction $1-\lambda$ of the new squared return. With $\lambda = 0.94$, the new shock receives weight 0.06 and the remaining 0.94 is carried from the previous variance estimate. The weighted-sum and recursive formulas describe the same object. The recursive formula shows the financial channel. This is the Lab 1 RiskMetrics choice $\lambda = 0.94$, so recent shocks matter but old shocks are not erased immediately. Volatility is a state variable. A large return shock enters tomorrow's variance estimate and then decays geometrically instead of disappearing after one observation. The volatility network should contain more edges than the return network because the dependent variable already stores past shock size. The sign of the shock is not the key object here. For example, a large negative return and a large positive return both raise r_{t-1}^2, so both raise the next volatility estimate. That is why

volatility can show a clearer pattern than signed returns: it keeps the size of the market move even when the direction is gone. The EWMA half-life is $\log(0.5)/\log(\lambda) \approx 11.2$ sampling periods. After that many observations, half of the original shock weight remains inside the variance estimate. The signed return may already look like noise, while the same shock can still move the volatility state.

For causality, the report uses the bivariate Granger setup from Lecture 4. In the equations below, Y_t is the receiving asset being forecast, X_t is the candidate source asset, h is the chosen lag length and ϵ_t is the forecast error. The unrestricted full model M_f contains two information blocks: the own lags of Y and the source lags of X. The restricted null model M_r removes the source block and keeps only the own lags of Y:

$$\begin{aligned} \text{Full model } M_f: \quad Y_t &= a_0 + a_1 Y_{t-1} + \cdots + a_h Y_{t-h} \\ &\quad + b_1 X_{t-1} + \cdots + b_h X_{t-h} + \epsilon_t, \end{aligned}$$

$$\text{Restricted model } M_r: \quad Y_t = a_0 + a_1 Y_{t-1} + \cdots + a_h Y_{t-h} + \epsilon_t,$$

$$H_0: \; b_1 = b_2 = \cdots = b_h = 0.$$

The coefficients $a_1, \ldots, a_h$ measure persistence in the receiving asset. The coefficients $b_1, \ldots, b_h$ are exactly the restrictions being tested. Under H_0, the restricted model is enough because the source asset adds no linear forecasting power once Y's own history is included. Rejecting H_0 means the full model has lower residual variance by enough to justify the extra source-lag block. The test is a forecast-comparison procedure. If X Granger-causes Y in this table, a linear forecaster that uses the past of both X and Y predicts Y_t better than a linear forecaster that uses only the past of Y. It does not mean that a shock to X would move Y under an intervention. The test has no trading experiment, no instrument, no order-flow channel and no control for common news. A rates shock, a dollar shock or a volatility-regime shift could move both assets and create a lagged forecasting relation. The edge records a narrower statement: the source asset contains incremental lagged information for this receiving asset in this sample and in this linear specification.

Tank *et al.* (2021) help us state the limitation precisely: many classical Granger procedures assume linear dynamics, even though real interactions can be nonlinear. In the notation above, our F-test can only find predictive content that enters through the linear block $b_1 X_{t-1} + \cdots + b_h X_{t-h}$. A missing edge records no detectable linear contribution at the selected lag in this sample. It does not rule out a stress-threshold channel, a nonlinear interaction or a relation that operates through a third asset. Tank *et al.* (2021) propose structured neural networks that search for nonlinear Granger structure by shrinking source-series inputs that do not help the forecast. We do not estimate that richer object here. Our tables should therefore be read as linear Granger evidence, not as a test of every possible nonlinear link.

Lag selection uses the VAR notation from Lecture 4:

$$y_t = A_1 y_{t-1} + \cdots + A_p y_{t-p} + u_t,$$

where y_t stacks the asset returns or volatilities, $A_1, \ldots, A_p$ are coefficient matrices and u_t is the vector of innovations. For every ordered pair, the implementation fits a bivariate VAR with no deterministic trend. In Python this is `statsmodels.VAR.select_order`

with `trend='n'`, the same deterministic specification as the `type='none'` setup used in the Python lab seen in class for Lecture 4 on VAR and Granger causality. AIC chooses p over lags up to 5 before the Granger F-test is evaluated. Mechanically, each extra lag adds one own-lag coefficient and one source-lag coefficient to the receiving-asset equation. AIC accepts that extra lag only when the improvement in residual fit is large enough to compensate for the larger parameter count. The advantage is that the lead-lag horizon is selected by a common likelihood penalty. The limitation is that a short sample can reward a lag that fits one historical episode rather than a channel that will survive in the next window. In simple terms, AIC is a way to choose how far back the test should look. It tries to balance two forces: fit the data well while avoiding too many lags for no reason. That is useful because we do not have to choose the lag by hand. It also means that large selected lags, especially in the monthly tables, need a cautious reading. They may describe a slow economic channel. They may also describe one episode that the model happened to fit well.

The four causality tables are decision tables, not effect-size tables. Rows are candidate source assets and columns are receiving assets. A non-zero entry is the AIC-selected lag when the 5% Granger test rejects the null. A zero means the source lags did not lower the residual variance enough to pass the nested F-test after the receiver's own lags were already included. The diagonal is omitted because an asset is not tested as a source for itself. The network figures redraw the same table decisions as arrows. They help identify source and receiver nodes visually. They do not add p-values, coefficient sizes or portfolio weights. A portfolio manager should use them as a first filter for follow-up models, not as direct trade signals. The number printed in a cell is therefore not the strength of the effect. It is only the AIC-chosen delay at which the source helped enough to pass the Granger test. For example, the weekly return table has SPY $\rightarrow$ ETH at the selected short lag. That means SPY's recent return helped forecast ETH's next return after ETH's own past was already included. It does not say that SPY mechanically moves ETH. It also does not say how large the return effect is. By contrast, SPY and XLK have a very high same-week correlation. The table does not give a SPY $\rightarrow$ XLK return arrow. They move together, but neither one leads the other in a useful way in this test.

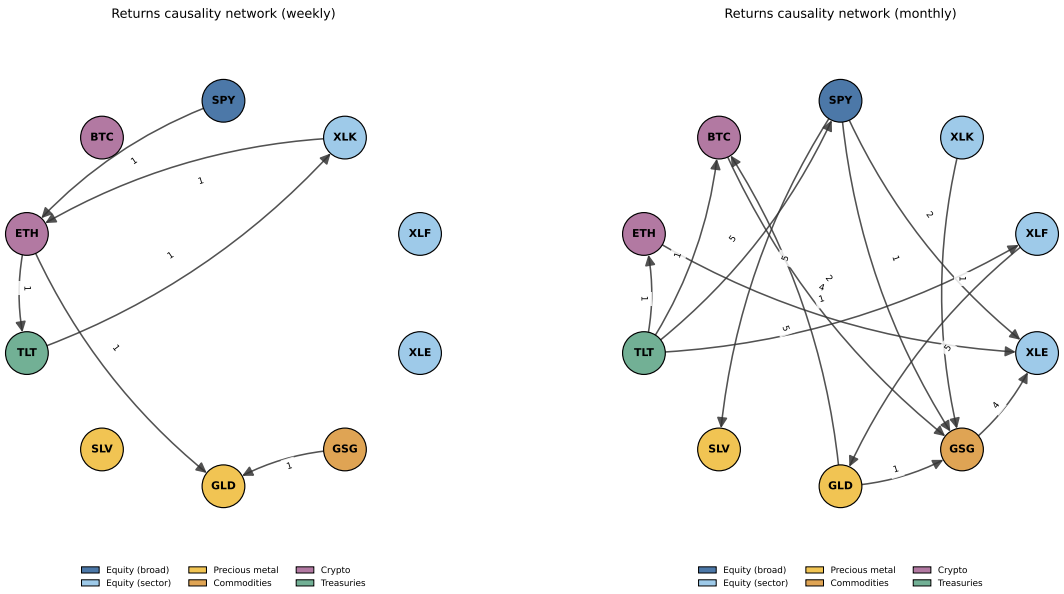
1b Return Causality Networks

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	0	0	0	0	0	1	0
XLK	0	-	0	0	0	0	0	0	1	0
XLF	0	0	-	0	0	0	0	0	0	0
XLE	0	0	0	-	0	0	0	0	0	0
GLD	0	0	0	0	-	0	0	0	0	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	0	0	0	1	0	-	0	0	0
BTC	0	0	0	0	0	0	0	-	0	0
ETH	0	0	0	0	1	0	0	0	-	1
TLT	0	1	0	0	0	0	0	0	0	-

Table 1: Pairwise Granger causality on weekly log returns. AIC-optimal lag in $[1, 5]$ if significant at the 5% level and 0 otherwise. *Notes:* Rows are source assets. Columns are receiver assets. A non-zero entry is the selected delay after the receiver's own past has already been included.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	2	0	5	1	0	0	0
XLK	0	-	0	0	0	0	1	0	0	0
XLF	0	0	-	0	5	0	0	0	0	0
XLE	0	0	0	-	0	0	0	0	0	0
GLD	0	0	0	0	-	0	1	5	0	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	0	0	4	0	0	-	0	0	0
BTC	0	0	0	0	0	0	2	-	0	0
ETH	0	0	0	4	0	0	0	0	-	0
TLT	5	0	1	0	0	0	0	1	1	-

Table 2: Pairwise Granger causality on monthly log returns. Entries as in the weekly returns table. *Notes:* The monthly sample is short. Lags near the edge of the search window point to possible macro channels rather than firm evidence of a stable trading delay.



(a) Weekly.

(b) Monthly.

Figure 1: Returns-causality networks. Edges run from cause to effect. Labels show the AIC-selected lag, not the economic magnitude of the effect.

Table 1 gives the most stable evidence in the return block: only 6 of 90 ordered pairs survive the 5% test. The table is sparse. The surviving links all use a short AIC lag. That short lag is important. It says that, when a return link appears, it is a quick lead-lag relation rather than a long delay. SPY and XLK lead ETH, which fits a simple market story: crypto can react after a broad equity or technology move changes risk appetite. GSG leading GLD is also plausible because a broad commodity move can come before a gold response when investors reprice inflation. The other links, such as ETH leading GLD and TLT or TLT leading XLK, are harder to read as stable economic mechanisms. They point to relations worth checking. They are not strong enough to become trading rules without a longer sample or a rolling check.

Figure 1 shows the same decisions without scanning the asset-by-asset matrix. The weekly graph has no dominant hub. If one market were moving first in a stable way, many arrows would leave that asset. Instead, the arrows are scattered. This is consistent with liquid markets: much of the news is priced into several assets during the same week. Once the receiver's own past returns are already in the regression, little extra cross-asset return information remains.

Table 2 is denser, with 14 edges. The monthly evidence is also less secure. The sample has only 23 monthly returns and the lag search allows horizons up to 5 months. TLT becomes a prominent source, with arrows into SPY, XLF, BTC and ETH. The macro channel is discount-rate transmission: long-duration Treasury prices reflect discount-rate and growth-news revisions, which can spill into equities and speculative assets. SPY also sends arrows to XLE, SLV and GSG, which fits a broad risk and commodity story for this particular two-year window. At the same time, several monthly arrows sit near the upper end of the lag search. With only 23 monthly returns, those long lags consume degrees of freedom quickly. A VAR at the maximum lag leaves 18 post-lag observations and estimates 11 coefficients per receiving-asset equation. AIC can still prefer that lag when residual fit improves. The F-test has little independent in-

formation left. The defensible reading is "possible macro-state transmission", not a robust market-timing rule. The monthly table therefore needs caution. AIC sometimes reaches for long lags because they fit the small sample better. That gives us a useful warning: the dense monthly network is a set of leads to investigate, not proof of strong predictability. For example, TLT leading SPY near the maximum allowed lag can be read as a possible rates channel: long Treasury prices may react first to monetary-policy expectations. Equities may adjust later through discount rates. But with only 23 monthly returns, that is a long delay to estimate. The table tells us where to look. It does not prove a dependable long-horizon trading delay.

The contrast with contemporaneous correlations clarifies the mechanism. Weekly SPY to XLK correlation is 0.93, GLD to SLV is 0.70 and BTC to ETH is 0.73. Those values show strong same-period comovement inside natural economic clusters. This means the assets often move together in the same week. That matters for portfolio risk, diversification and covariance estimation. The Granger tables ask a harder question. Once an asset's own past returns are included, does another asset's past return still help predict it? High same-week correlation does not prove that. Two assets can move together because they react to the same news at the same time. That does not mean one asset usefully leads the other. The practical conclusion is simple: the evidence for covariance is clear. The evidence for cross-asset return forecasting is weak.

One event helps interpret the crypto links. In January 2024, the SEC approved spot Bitcoin ETPs (U.S. Securities and Exchange Commission, 2024). This happened during our sample and made Bitcoin easier to trade through regulated exchange products. For example, the weekly table reports SPY $\rightarrow$ ETH at the selected short lag. One possible interpretation is a broad risk channel: equity-market news moves first, then crypto adjusts as investors change risk exposure. The SEC event makes that interpretation plausible for this period. It does not prove the link or turn it into a trading rule.

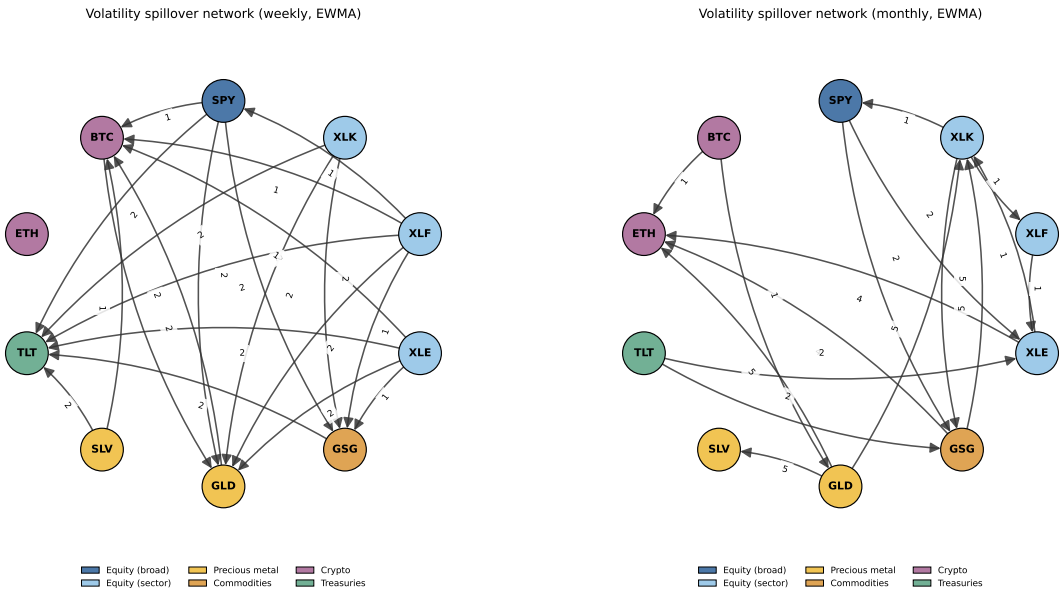
1c Volatility Spillover Networks

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	0	2	0	2	1	0	2
XLK	0	-	0	0	2	0	2	0	0	2
XLF	1	0	-	0	2	0	1	1	0	2
XLE	0	0	0	-	2	0	1	1	0	2
GLD	0	0	0	0	-	0	0	2	0	0
SLV	0	0	0	0	0	-	0	1	0	2
GSG	0	0	0	0	0	0	-	0	0	2
BTC	0	0	0	0	2	0	0	-	0	0
ETH	0	0	0	0	0	0	0	0	-	0
TLT	0	0	0	0	0	0	0	0	0	-

Table 3: Pairwise Granger causality on weekly EWMA volatility. Entries as in the returns tables. *Notes:* The test is applied to EWMA volatility states. A non-zero cell means volatility in the row asset helps forecast volatility in the column asset after the column asset's own volatility history is used.

	SPY	XLK	XLF	XLE	GLD	SLV	GSG	BTC	ETH	TLT
SPY	-	0	0	2	0	0	2	0	0	0
XLK	1	-	1	0	0	0	5	0	0	0
XLF	0	0	-	1	0	0	0	0	0	0
XLE	0	1	0	-	0	0	0	0	4	0
GLD	0	5	0	0	-	5	0	0	5	0
SLV	0	0	0	0	0	-	0	0	0	0
GSG	0	5	0	0	0	0	-	0	4	0
BTC	0	0	0	0	1	0	0	-	1	0
ETH	0	0	0	0	0	0	0	0	-	0
TLT	0	0	0	2	0	0	2	0	0	-

Table 4: Pairwise Granger causality on monthly EWMA volatility. Entries as in the returns tables. *Notes:* Monthly volatility mixes EWMA memory with calendar aggregation. Long delays may describe slow risk regimes. They may also reflect limited monthly data.



(a) Weekly.

(b) Monthly.

Figure 2: EWMA volatility-spillover networks. Edges run from source volatility to receiver volatility. Labels show the AIC-selected lag.

Volatility is much more connected than returns. Tables 3 and 4 contain 21 weekly edges and 17 monthly edges, compared with 6 weekly return edges and 14 monthly return edges. That gap is the empirical footprint of a second moment that carries memory. EWMA smooths squared returns, so a shock in one market can remain visible in the volatility state after the signed return itself has faded. This is the main difference between the two networks. Returns ask, "which way did prices move?" Volatility asks, "how nervous is the market?" Nervousness tends to last longer than the sign of one return, so it is easier for volatility to spill from one asset group to another.

Table 3 has a recognisable risk-channel structure. SPY, XLK, XLF and XLE volatility lead volatility in GLD, GSG, BTC and TLT. Many of these weekly links use short AIC lags. That timing is believable: an equity shock can appear first in stock volatility, then

show up in hedges, commodities, Treasuries or crypto after investors rebalance. TLT appears mainly as a receiver rather than a source in the weekly volatility table. The table is therefore consistent with a simple risk story: stress starts in the equity block and then spreads into assets used for hedging or for expressing broader macro views.

Figure 2 adds a structural reading that the table alone hides. The weekly volatility graph has clearer source-to-receiver blocks than the return graph. Equity volatility is especially integrated: SPY to XLK volatility correlation is 0.71, SPY to XLF is 0.87 and SPY to XLE is 0.92. The relevant mechanism is a conditional-risk update. If the weekly table contains an edge $i \to j$ at lag ℓ, then $\sigma_{i,t-\ell}$ is a candidate feature for forecasting $\sigma_{j,t}$ in a multivariate volatility model. That feature would change the next-period variance estimate for asset j. Through the covariance matrix it would affect VaR, expected shortfall, margin usage or risk-budget allocations. For risk management, the implication is direct. When sector volatility leads Treasury, commodity or crypto volatility, the risk forecast for the receiving asset should condition on the earlier sector shock.

Table 4 shifts the emphasis toward commodities, metals and crypto. GLD and GSG become visible sources, while ETH is mostly a receiver. The aggregation mechanism changes the object being tested. At the monthly frequency, the shock being measured is less about the first trading response and more about a regime that persists through the month: inflation concern, commodity repricing or a broader speculative-risk adjustment. The limitation comes from smoothing and sample size. Monthly EWMA values carry past shocks forward. With only 22 volatility observations the test can overstate the stability of slow-moving relationships. Here AIC again deserves attention: several monthly volatility arrows use long selected lags, often near the maximum allowed value. That can point to slow inflation or real-rate channels, but it can also be a small-sample effect. The main evidence is not any single arrow. It is the contrast across objects: return levels are hard to predict, while volatility regimes propagate across assets more clearly.

The Exercise 1 message is practical. The return tables are useful for finding possible lead-lag stories. Most of those stories are weak. The volatility tables are more convincing because risk states have memory. AIC applies the same lag-selection rule across pairs. It does not decide whether the story is economically strong. We still need to ask whether the selected lag makes sense. A short weekly lag is more credible because the weekly sample has many more observations and the mechanism is natural: an equity-volatility shock can be noticed first in SPY, XLK, XLF or XLE and then show up shortly after in hedges, commodities, Treasuries or crypto as investors rebalance. That is why weekly volatility links such as equity-sector volatility leading GLD, GSG, BTC or TLT are easier to interpret. A lag near the maximum allowed monthly horizon is different. With only 23 monthly returns or 22 monthly volatility observations, a lag near 5 leaves very little data after lagging. TLT leading SPY near that maximum lag in returns and GLD or GSG leading other assets at long lags in volatility, may point to a slow rates, inflation or commodity channel. These patterns are not enough for portfolio decisions. They should be checked in a longer sample, in rolling windows or with another forecasting model.

2 S&P 500 Forecasting

The second exercise follows the HW1 forecasting brief: use the assigned Goyal-Welch predictors, their lags and target lags to compare neural networks, LSTMs and Gaussian-process models. For this team the assigned predictors are dp, svar and bm. The three targets separate three different forecasting questions:

$$r_{t+1} = \ln P_{t+1} - \ln P_t, \qquad p_{t+1} = \ln P_{t+1}, \qquad d_{t+1} = \mathbf{1}\{r_{t+1} > 0\}.$$

The first target, r_{t+1}, asks for next month's log return. It removes the known price level and leaves only the new payoff. The second target, p_{t+1}, asks for next month's log price level. That target is easier to fit because p_t already carries most of the information needed to approximate p_{t+1} in a persistent index. The third target, d_{t+1}, records only the sign of the next return. It equals one for an up month and zero otherwise. Features use up to 3 lags of the target-relevant series and up to 3 lags of dp, svar and bm, giving 12 candidate features before selection. The chronological split is 60% train, 15% validation and 25% test. After date filtering and NA removal the panel has 1140 monthly observations. Lag construction and next-month target alignment leave 1137 usable rows: 682 for training, 170 for validation and 285 for testing. Feature scaling and feature selection are fitted before the test block is touched. The test score is a forecast on later calendar months, not a random split that mixes future regimes into the training set.

The feature-selection layer is deliberately conservative. Return and price regressions use `LassoCV`. Direction uses an ℓ_1-penalised `LogisticRegressionCV`. The inner validation folds use `TimeSeriesSplit`, so a model is never validated on data that occur before its training data. This prevents a common financial leakage mechanism: random folds would let coefficients learn from recessions, crises or rallies that occur after the months they are asked to forecast.

The model grids are small because the experiment asks whether the assigned predictors contain stable signal, not whether a very large search can overfit the validation block. The NN grid compares one-layer and two-layer MLPs with ReLU and tanh activations. The GP grid compares RBF, Matern, RationalQuadratic and additive kernels that include linear and white-noise components. The LSTM grid varies width, depth and sequence length, then uses early stopping on an internal validation slice. The advantage is comparability: every model sees the same chronological information set. The limitation is also explicit: a larger hyperparameter search could change validation rankings, but it would not remove the low signal-to-noise ratio of monthly returns.

The Python lab seen in class for Lecture 5 on NNET, GaussProc and LSTM forecasting defines the scoring convention. To keep the tables readable, the report writes that convention in mathematical form:

$$\begin{aligned}
\text{MSE} &= \frac{1}{N}\sum_{t=1}^{N}(y_t - \widehat{y}_t)^2, \\
\text{NRMSE} &= \sqrt{\frac{\text{MSE}}{\sum_{t=1}^{N}(y_t - \widehat{\mu}_y)^2}}, \\
\text{pcorrect} &= 100(1 - \text{NRMSE}).
\end{aligned}$$

Here N is the number of test observations, y_t is the realised regression target, $\widehat{y}_t$ is the model forecast and $\widehat{\mu}_y$ is the sample mean used only to scale the scoring error. The denominator uses a total-sum-of-squares scale. Dividing by that scale turns RMSE into an error relative to the dispersion of the target, which is why the transformed pcorrect values sit near the upper end of the scale. This pcorrect is not the same object as direction accuracy. It is computed as $100(1 - \text{NRMSE})$, so it measures a scaled regression error rather than the probability of getting the sign right. The convention is kept because it matches the course lab and the original solution values.

The neural-network, LSTM and GP models map into the notation from Lecture 5. For a recurrent model, x_t is the vector of predictors observed at time t, h_t is the hidden state carried forward from the past, W_h and W_x are weight matrices, b is a bias and σ is the activation function:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b), \qquad y_t = \phi(W_y h_t + c).$$

The hidden state h_t is the memory channel. Information from earlier months can reach the forecast only if it is encoded into that state and carried forward by the recurrence. Lecture 5 shows how the LSTM makes this channel more selective by adding gates. For example,

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f), \qquad i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i).$$

The forget gate f_t decides how much old state to retain and the input gate i_t decides how much new information to write. That design matches a financial series in which a macro or risk regime can last for several months and then fade. The GP is different. It does not carry a hidden state. It defines similarity through a kernel. Its point forecast is the posterior mean

$$\bar{f}_* = \mathbb{E}[f_* | X_*, X, y] = K(X_*, X)\,[K(X, X) + \sigma_n^2 I]^{-1} y.$$

Here X and y are the training predictors and targets, X_* is a test point, $K(\cdot, \cdot)$ is the kernel matrix and σ_n^2 is the noise variance. Lecture 5 presents the GP prediction as a posterior mean under a kernel. The prediction weights training outcomes through $K(X_*, X)$, so the kernel defines which historical months are allowed to influence the test forecast. This is a strong structure for the price target, where nearby price levels carry a lot of information. It is weaker for returns because similar predictors can be followed by very different next-month shocks.

Arratia *et al.* (2020) provide the methodological link used in this exercise. They show why neural networks and Gaussian processes are related model families in the wide-network limit, where random network weights imply a GP prior over functions. They also use monthly S&P 500 prices and price lags in a regression example. The mechanism is kernel covariance: GP predictions weight observed training targets by similarity $K(X_*, X)$, while a finite neural network learns weights directly by optimisation. This supports comparing NN and GP forecasts in the same exercise. It does not prove that either model has a tradable one-month return signal.

2a Feature Selection and Target Design

The L1 feature-selection step keeps bm_lag1, bm_lag2 and log_ret_lag3 for returns. For price it keeps log_price_lag1, dp_lag1, svar_lag1, svar_lag3 and bm_lag2. For direction

it keeps 11 variables. That deserves caution: a classifier with many weak predictors can fit small historical imbalances without learning a stable sign rule. The selected variables are inputs to the forecasting experiment, not a stable economic model of the equity premium. The held-out test block decides whether those inputs improve forecasts after the feature-selection step has already used the training data.

The selected features are still informative about the statistical problem. For returns, book-to-market lags enter together with a lagged return, which is consistent with valuation carrying slow-moving information but not necessarily short-run timing power. For prices, `log_price_lag1` is selected, so the model is allowed to use the strongest possible state variable: yesterday's level. Direction keeps almost the whole lag block. That breadth is a warning sign rather than a success. If many weak variables are needed to classify a binary monthly sign, the boundary is likely fragile and should be evaluated by MCC rather than accuracy alone. Put differently, feature selection already tells us that the easy information is in the lagged target. The price model keeps yesterday's price because that is highly useful. The return model keeps only a small set because monthly returns contain much less stable signal. The direction model keeps many features. That is not automatically good. It may mean that the classifier is searching hard for a weak pattern.

2b Return Forecasts

Model	Best configuration	RMSE	NRMSE	pcorrect
NN	2 layers, $(128, 64)$, ReLU	0.0526	0.0701	92.99
GP	RBF + WhiteNoise	0.0445	0.0592	94.08
LSTM	1 layer, 64 units, T=6	0.0472	0.0629	93.71
Baseline	training mean	0.0445	0.0593	94.07

Table 5: Return-target test performance, scoring convention from the Python lab seen in class for Lecture 5 ($n = 285$). *Notes:* RMSE is in monthly log-return units. NRMSE puts the same error on the course scale. pcorrect is built from NRMSE. It is not the probability of calling the sign correctly.

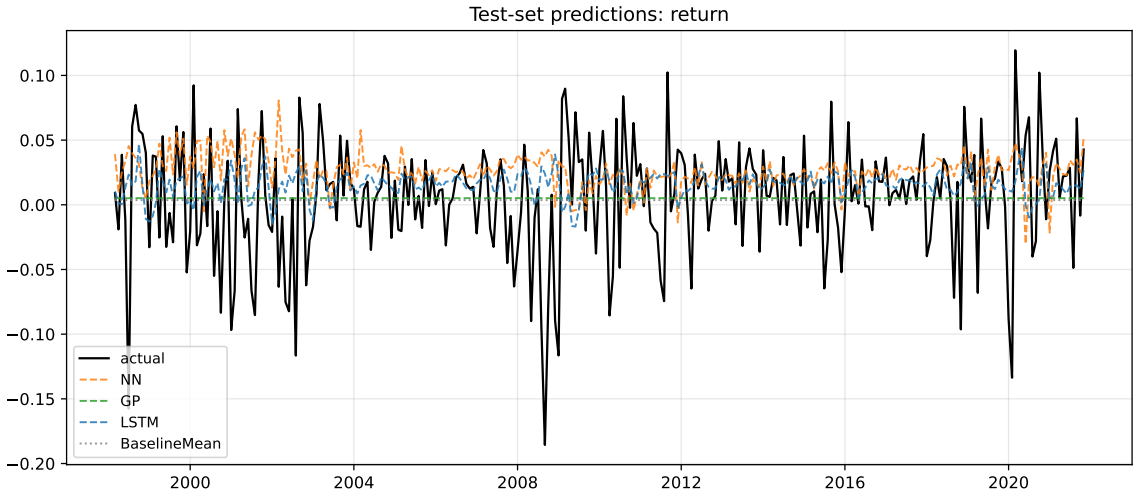


Figure 3: Test-set predictions of next-month log return.

The return task is the most direct financial forecasting problem because it asks for the next movement rather than the next level. The expectation from the Lecture 5 forecasting discussion and from the predictability literature is modest. If return signals are weak, a flexible model should not dominate a mean forecast by much.

Table 5 matches that expectation. RMSE is in monthly log-return units, while NRMSE and pcorrect use the scaling from the Python lab seen in class for Lecture 5 described above. The GP has the best ML score. Its NRMSE, 0.0592, is almost identical to the mean baseline NRMSE, 0.0593. Economically, this is the central result of Exercise 2b. The flexible models find patterns in the training and validation blocks, yet those patterns do not translate into a meaningful improvement over an unconditional expected return. Goyal and Welch (2008) report the same broad difficulty for standard equity-premium predictors.

Figure 3 supports the table reading. The realised return series has sharp positive and negative moves, while the fitted lines are much smoother. The model is estimating a conditional mean, while the realised payoff is dominated by news shocks that were not in the predictor set. Adding a layer or changing the kernel would not reveal information that the lagged variables do not contain. These predictors, these lags and this chronological split do not provide a tradable one-month return forecast. This does not mean the models failed technically. It means the target is very hard. For monthly returns, the best forecast under squared-error loss is often close to the historical average. The GP being almost tied with the mean baseline is therefore not a dramatic win. It is also not surprising. The data are telling us that most of the next month's return is new information, not something already contained in dp, svar, bm or a few return lags.

2c Log-Price Forecasts

Model	Best configuration	RMSE	NRMSE	pcorrect
NN	1 layer, 64 units, ReLU	0.3272	0.0466	95.34
GP	custom RBF + Linear + WhiteNoise	0.0795	0.0113	98.87
LSTM	1 layer, 64 units, T=6	1.2423	0.1770	82.30
Baseline	training mean	3.8198	0.5442	45.58

Table 6: Log-price target test performance, scoring convention from the Python lab seen in class for Lecture 5 ($n = 285$). *Notes:* The target is a log-price level. A strong score can therefore come from persistence in yesterday's price even when the implied return forecast adds little.

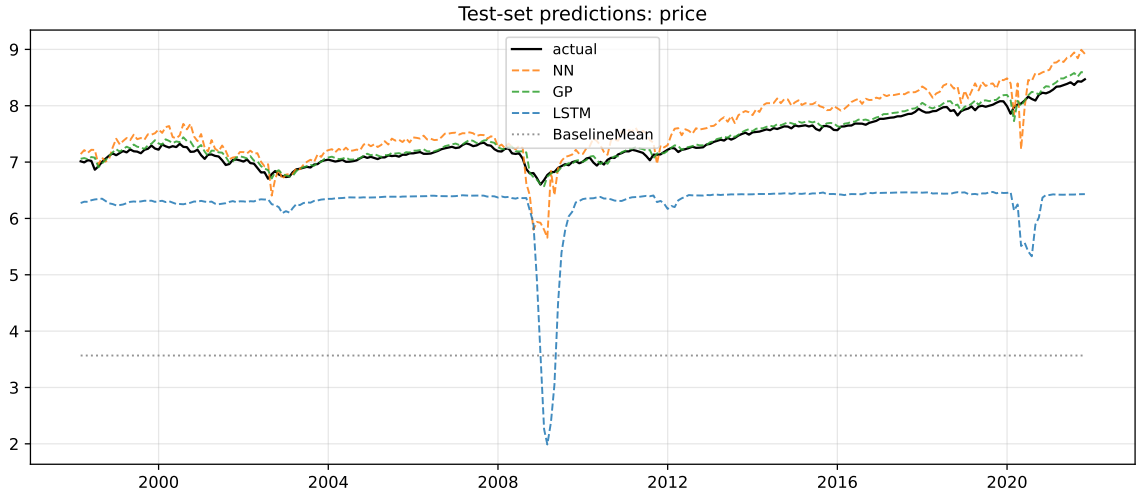


Figure 4: Test-set predictions of next-month log price.

The price target is easier on the plot because $\log P_t$ is highly persistent. A good price forecast can be obtained by staying close to the last observed level, even if the model has little information about the next return.

Table 6 answers a different question from the return table. The GP has the best Exercise 2 NRMSE at 0.0113 and 98.87% pcorrect. That is a strong statistical fit. It is not evidence of a strong equity-premium signal. The selected lagged price feature gives the model direct access to the level. The custom RBF plus linear kernel is well suited to extrapolating a smooth, persistent state. The LSTM is weaker because the sequence window and training objective do not give it the same simple level-following mechanism.

Figure 4 illustrates the distinction. The forecasts track the level of the index far better than the return forecasts track monthly innovations. For a risk or asset-allocation question, however, a precise forecast of $\log P_{t+1}$ changes a decision only if it also improves the implied return forecast. The price figure shows level persistence and kernel smoothing, not evidence that the model has found exploitable expected returns. Put simply, predicting the log price is close to saying: tomorrow's level will look a lot like today's level. That is usually a good statement for a stock index. It creates a high table score. It is a weaker finance result than it appears. Investors care about the change from today to tomorrow, not only about the level tomorrow. As a simple example, if the index is at level P_t today, a model that forecasts a level close to P_t next month will often have a small price error. But the investment question is whether P_{t+1} is meaningfully above or below P_t. The price score can be good even when the return forecast is not useful.

2d Direction Forecasts

Model	Best configuration	accuracy	MCC
NN	1 layer, 128 units, tanh	0.6070	-0.0571
GP	RatQuad + WhiteNoise	0.6246	0.0000
LSTM	2 layers, 32 units, T=6	0.6000	-0.0423
Baseline	training majority class	0.6246	0.0000

Table 7: Direction-target test performance ($n = 285$). *Notes:* Accuracy can look respectable when up months are common. MCC adds the missing balance check by asking whether the model separates up months from down months.

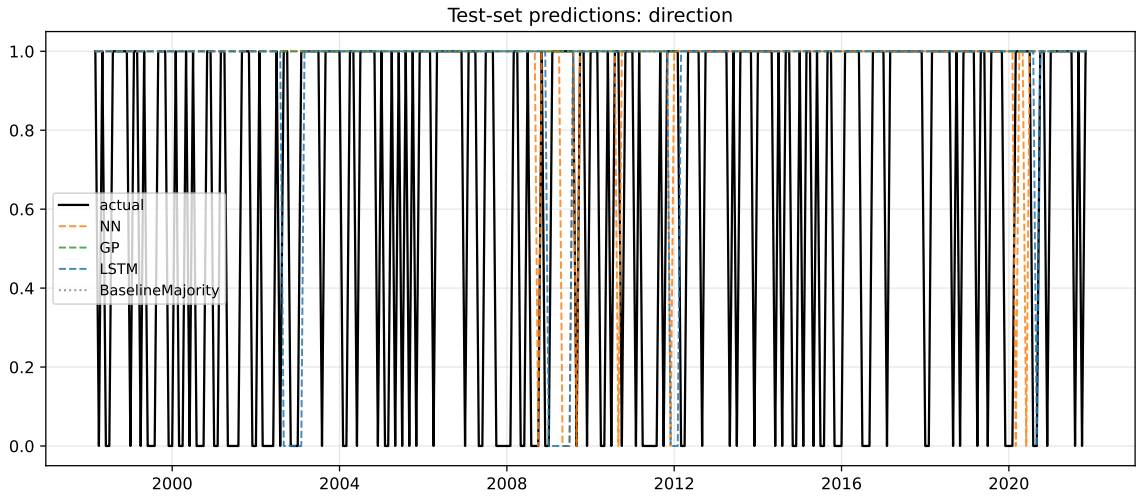


Figure 5: Test-set predictions of next-month direction.

Accuracy alone is misleading because the test set has many up months. A model can obtain a respectable accuracy simply by predicting "up" most of the time. Table 7 shows the problem. The GP and the majority baseline both reach 0.6246 accuracy with MCC equal to 0.0000, which means they are effectively majority-class forecasts. The NN and LSTM make non-constant decisions. Yet their MCCs, -0.0571 and -0.0423, are negative. They fail to beat the majority rule and make the wrong trade-off between false positives and false negatives.

Figure 5 displays the forecasts as discrete sign calls rather than regression errors. A direction model can look acceptable in a table if the market rises often enough. The figure shows whether the model is actually switching states. In this sample the switching does not correspond to a stable classification boundary. Accuracy answers how often the sign was right. MCC asks whether the classifier separates up months from down months in a balanced way. The latter is the more relevant diagnostic for a trading or allocation signal. This is why the direction result is weaker than the accuracy number suggests. The market is positive in many test months, so a lazy rule can look decent. MCC is stricter. It asks whether the model is useful on both sides of the problem: up months and down months. By that standard, the full-feature NN and LSTM do not give reliable sign forecasts. The GP mostly defaults to the majority class. For example, a model that always predicts "up" can score well if up months are common. It gives no

help in identifying down months. This is why the GP can match the majority baseline in accuracy while having MCC equal to zero. The MCC value says the classifier is not really separating the two classes.

2e Sensitivity and Target Comparison

Target	Dropped	NN	GP	LSTM
return	–	0.0701	0.0592	0.0629
return	dp	0.0615	0.0592	0.0601
return	svar	0.0615	0.0592	0.0601
return	bm	0.0658	0.0592	0.0599
price	–	0.0466	0.0113	0.1770
price	dp	0.0214	0.0135	0.1474
price	svar	0.0378	0.0075	0.1357
price	bm	0.0321	0.0067	0.1475
direction is reported as accuracy				
direction	–	0.6070	0.6246	0.6000
direction	dp	0.6246	0.6246	0.6246
direction	svar	0.6035	0.6246	0.6246
direction	bm	0.6211	0.6246	0.6175

Table 8: Sensitivity analysis. Regression cells report NRMSE. Direction cells report accuracy. Each ablation removes all lags of the indicated predictor. *Notes:* The row with – keeps the full feature set. Each named row removes all lags of that predictor before feature selection and model refitting.

Table 8 is a small ablation study. It asks whether the forecast comes from the economic predictors themselves or from the autoregressive part of the design matrix. In finance terms, dp and bm are valuation ratios. They move slowly because dividends, book values and prices adjust at different speeds. svar is a risk-state variable. It reacts to market stress and should matter most when variance risk carries a premium. In ML terms, all three variables are weak, persistent and correlated with the lagged target. An ablation can improve performance by removing a coordinate that adds estimation variance faster than it adds stable signal, even when the removed variable has a sensible economic interpretation.

The table mixes two scoring conventions, so the rows need careful reading. Return and price cells report NRMSE, a regression-error scale. Direction cells report accuracy, a sign-hit rate. In the return block, dropping any one of dp, svar or bm leaves the GP unchanged and improves the NN/LSTM slightly. That pattern says the exogenous predictors are not carrying robust short-run return information. They may still be economically meaningful at lower frequency. The one-month supervised forecast does not extract stable incremental signal from them.

The return block gives the clearest warning against over-reading flexible models. Removing dp, svar or bm does not create a clear return-forecasting gain. The GP barely moves because the RBF kernel is effectively smoothing around a mean return in a low

signal-to-noise problem. The NN and the LSTM often improve after a variable is removed, which is a variance effect rather than a finance discovery. A monthly equity return is mostly news. Once the conditional mean is small relative to the innovation variance, extra predictors can raise estimation error faster than they add signal. This is the standard bias-variance tradeoff in a financial setting: the model class is flexible enough to fit patterns, while the economic target does not provide enough stable conditional mean to make those patterns valuable out of sample.

The price block behaves differently because the target is dominated by the state variable $\log P_t$. The GP reaches its best price NRMSE when bm is removed, with NRMSE 0.0067. This does not mean that book-to-market has no economic content. It means that, in this supervised task, the lagged price level already carries the low-frequency trend that the GP needs. Removing a slow valuation ratio can reduce collinearity in the kernel geometry. The model then compares test points mainly through the persistent price coordinate rather than through a crowded set of slow-moving predictors. Price forecasting looks impressive in the table because the model is learning level persistence much more than equity-premium predictability.

The direction block needs extra care. The table reports accuracy there because NRMSE is not a classification metric. Accuracy near the majority baseline can come from calling most months positive, which is exactly why Table 7 also reports MCC. The sensitivity rows show that dropping a predictor does not produce a stable classification rule. From a market perspective the horizon mismatch is the mechanism. A valuation signal can affect long-horizon expected returns and still be weak for one-month signs. From an ML perspective the classification boundary is trying to separate many small positive returns from many small negative returns using sluggish predictors. That is a hard problem because the label throws away magnitude information and keeps only the sign. The useful lesson from the sensitivity table is not that one variable is "bad" in general. It is narrower. In this particular one-month forecasting exercise, removing some assigned predictors often makes the model simpler and slightly more stable. That supports the main interpretation: the Goyal-Welch variables may contain slow economic information. They do not deliver a clean short-run forecasting edge here.

The target comparison is not a ranking by headline fit. Price is easiest to fit because it has a near-unit-root level. Return is the economically direct target because it asks for the new payoff. Direction is appealing for trading language, although it is statistically fragile under class imbalance. The practical conclusion is that model flexibility helps when the target contains a strong state variable. It is far less reliable when the target is the conditional mean of monthly returns. So the answer to "which target works best?" has two parts. If we only look at table fit, the price target wins. If we ask the harder finance question, returns and directions matter more. Those results are weak. This distinction keeps the conclusion honest: the models can fit a persistent price level, but they do not uncover a strong one-month equity-return signal.

3 ARMA-GARCH Benchmarks

The weak return results in Exercise 2 make a classical benchmark necessary. Exercise 3 compares the ML models with ARMA-GARCH baselines on the same test block. The

Lecture 3 time-series decomposition separates a series into a deterministic component, a predictable conditional mean and an innovation:

$$X_t = D_t + Y_t, \qquad Y_t = \mathbb{E}(Y_t|\mathcal{F}_{t-1}) + a_t, \qquad \sigma_t^2 = \mathrm{Var}(Y_t|\mathcal{F}_{t-1}) = \mathbb{E}(a_t^2|\mathcal{F}_{t-1}).$$

Here D_t is the deterministic part, Y_t is the stochastic component, $\mathcal{F}_{t-1}$ is the information set available before date t, a_t is the unexpected innovation and σ_t^2 is the conditional variance. The decomposition says that a time series model has two jobs: forecast the conditional mean and describe the remaining conditional risk. In finance those two jobs feed different decisions: the mean forecast drives expected payoff, while the variance forecast drives capital, margin and drawdown control. A model can fail on the mean and still improve the risk estimate. For log returns, the Lecture 3 ARMA model is

$$r_t = \phi_1 r_{t-1} + \cdots + \phi_p r_{t-p} + a_t + \theta_1 a_{t-1} + \cdots + \theta_q a_{t-q}.$$

The ϕ terms use lagged returns, while the θ terms use lagged forecast errors. If return predictability is weak, this mean equation should not beat the best ML model by a large economic margin. The Lecture 3 GARCH model describes the innovation variance:

$$a_t = \sigma_t \varepsilon_t, \qquad \sigma_t^2 = \omega + \sum_{i=1}^{p} \alpha_i a_{t-i}^2 + \sum_{j=1}^{q} \beta_j \sigma_{t-j}^2, \qquad \varepsilon_t \sim N(0,1).$$

In this equation, ω is the long-run variance intercept, the α_i terms measure how strongly recent squared shocks move volatility and the β_j terms measure how much past volatility carries forward. The sum of the fitted α and β terms is the persistence diagnostic because it measures how much of a variance shock remains in the next period's variance recursion. Values near one mean that volatility shocks fade slowly. This gives a concrete expectation before Table 9. The ARMA part should be close to the ML models on returns, while the GARCH part should capture the persistence that also appeared in the EWMA volatility networks.

3a Model Orders and Volatility Persistence

Both ARMA-GARCH variants use the same monthly log-return series as Exercise 2. The fit block combines the training and validation periods. The test block runs from 1998-03 to 2021-11 and contains 285 observations. ARMA order is selected by AIC over $p, q \in \{0, \ldots, 5\}$ with `pmdarima.auto_arima`. We set no seasonal component because the target is a monthly equity return. We use no differencing because log returns are already the stationary object in the Lecture 3 setup. The GARCH(1,1) layer is fitted on the ARMA residuals, so the mean equation and the volatility equation each receive the part of the series they are designed to model.

For the Yeo-Johnson variant, the transformation is estimated only on the combined training and validation block. The ARMA-GARCH model is fitted on the transformed returns, forecasts are inverse-transformed and all scoring is done back in original return units. Yeo-Johnson is used because returns can be negative. Box-Cox would require strictly positive inputs. A shift could make returns positive. That shift would change the scale and make the transformed likelihood less tied to the original financial object.

One-step test forecasts use fixed parameters and expanding information. The implementation extends the fitted ARMA model through the test path using the `apply` method with `refit=False`, then reads the fitted one-step values for the test block. The model is not re-estimated at every test date. It uses realised past test returns to update the ARMA state. It does not use future returns to estimate parameters. The GARCH equation supplies the conditional-variance path and the persistence diagnostic. The point forecasts for return, price and direction come from the ARMA conditional mean, with $\widehat{p}_{t+1} = p_t + \widehat{r}_{t+1}$ and direction obtained by thresholding $\widehat{r}_{t+1}$ at zero.

The raw-return baseline selects ARMA(2,2) with AIC -2482.78 and GARCH persistence 0.984. The Yeo-Johnson variant selects ARMA(0,1) with transformed-scale AIC 2416.10 and persistence 0.982. The two AIC values should not be compared directly because the likelihoods are evaluated on different return scales. The model-order comparison is still informative: the transformation lets the conditional-mean equation use fewer ARMA terms, which reduces the number of serial-correlation patterns it can chase in sample. Persistence close to one is common in equity volatility. The plain model estimates $\alpha + \beta = 0.984$ and the transformed model estimates $\alpha + \beta = 0.982$. Both are near the integrated boundary. Financially, that means the variance forecast responds to a shock and then releases it slowly across later months. From an ML benchmarking view, this is a low-dimensional model with a strong inductive bias: it does not try to learn a broad nonlinear mean function. It concentrates capacity on volatility persistence, which is the part of the data that was visibly stable in Exercise 1. Numerically, the plain persistence value 0.984 means that almost all of a volatility shock is carried into the next monthly variance estimate. A bad month does more than affect that month. It also raises the risk forecast for the following months, then fades slowly. The AIC result also helps explain why the transformed version is attractive. The raw returns need ARMA(2,2), while the transformed series only needs ARMA(0,1). In simple words, Yeo-Johnson makes the mean model cleaner. It does not create a large return signal. It reduces the chance that the ARMA part is fitting small, noisy wiggles in the training sample.

3b ML Versus ARMA-GARCH

Target	Model	RMSE	NRMSE	pcorrect	accuracy	MCC
return	NN	0.0526	0.0701	92.99		
return	GP	0.0445	0.0592	94.08		
return	LSTM	0.0472	0.0629	93.71		
return	BaselineMean	0.0445	0.0593	94.07		
return	ARMA-GARCH (plain)	0.0451	0.0601	93.99		
return	ARMA-GARCH (YeoJohnson)	0.0444	0.0591	94.09		
price	NN	0.3272	0.0466	95.34		
price	GP	0.0795	0.0113	98.87		
price	LSTM	1.2423	0.1770	82.30		
price	BaselineMean	3.8198	0.5442	45.58		
price	ARMA-GARCH (plain)	0.0451	0.0064	99.36		
price	ARMA-GARCH (YeoJohnson)	0.0444	0.0063	99.37		
direction	NN				0.6070	-0.0571
direction	GP				0.6246	0.0000
direction	LSTM				0.6000	-0.0423
direction	BaselineMajority				0.6246	0.0000
direction	ARMA-GARCH (plain)				0.6000	0.0759
direction	ARMA-GARCH (YeoJohnson)				0.6246	0.0627

Table 9: Exercise 2 and Exercise 3 comparison on the same test block ($n = 285$). *Notes:* Return and price rows use regression errors. Direction rows use sign classification measures. Blank cells keep those two scoring problems separate while preserving the same chronological test block.

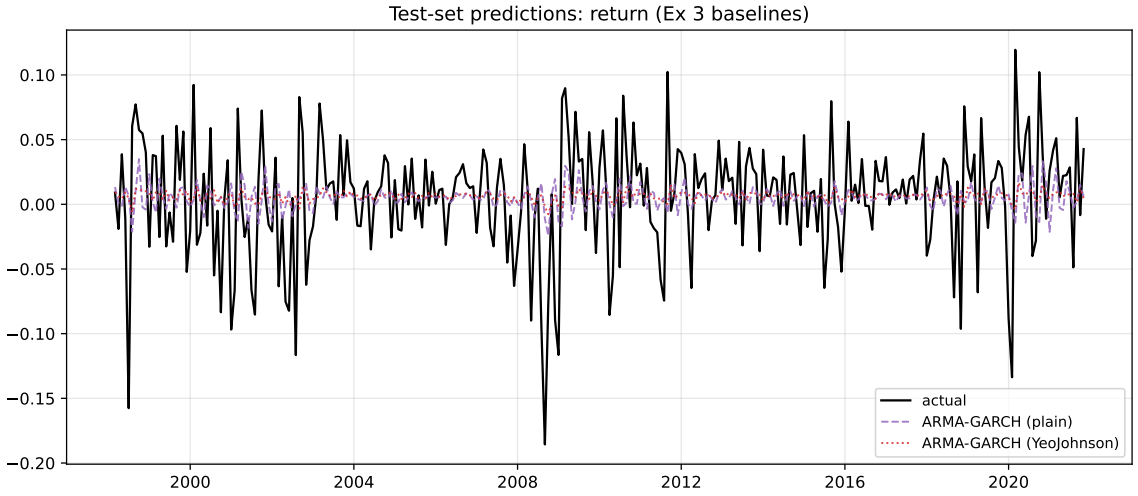


Figure 6: ARMA-GARCH return forecasts.

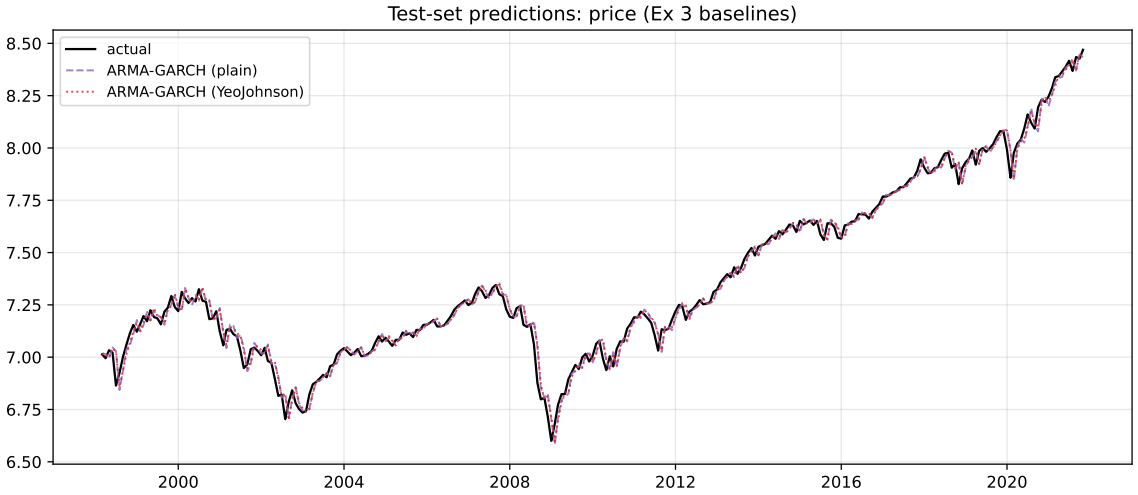


Figure 7: ARMA-GARCH log-price forecasts.

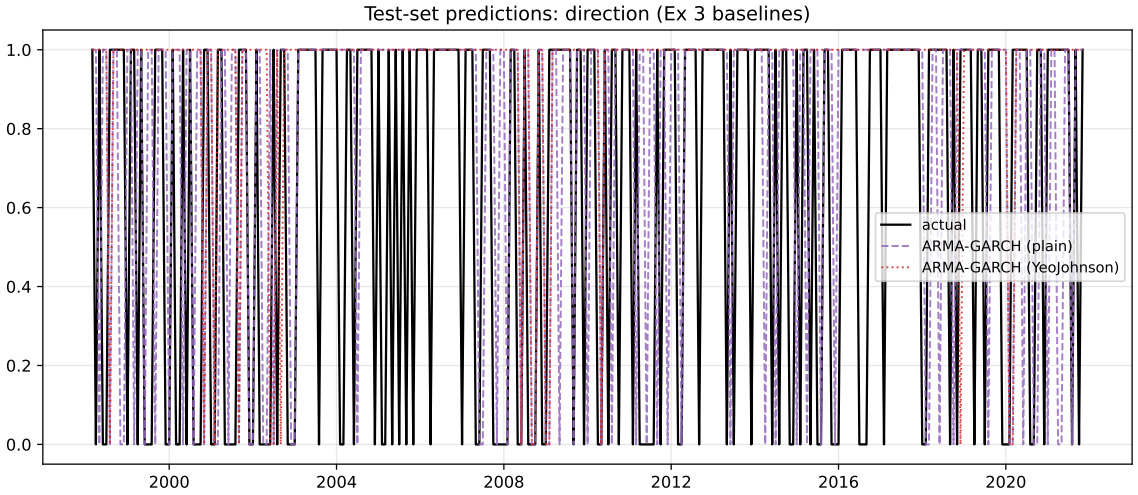


Figure 8: ARMA-GARCH direction forecasts.

Table 9 is a joint diagnostic table rather than one single leaderboard. The return and price blocks report RMSE, NRMSE and pcorrect. The direction block reports accuracy and MCC because the target is a binary sign. The empty cells are intentional. They prevent a regression metric from being compared with a classification metric.

On returns, Yeo-Johnson ARMA-GARCH has NRMSE 0.0591, while the Exercise 2 GP has NRMSE 0.0592. The difference is economically tiny. It is a practical tie because the improvement is small relative to the scale of monthly return shocks. The plain ARMA-GARCH result, 0.0601, also sits very close to the mean benchmark. One-month expected returns are difficult to forecast with either flexible ML models or classical conditional-mean models. Extra functional flexibility does not help much when the target has a low signal-to-noise ratio and the stable component is close to an unconditional mean.

Figure 6 is the visual check behind that conclusion. The forecast path is smoother than the realised return path. Large realised moves remain mostly innovations. This is the right picture for a model that captures a modest conditional mean but cannot anticip-

ate the news shock itself. A profitable trading interpretation would require the forecast to survive rolling re-estimation, transaction costs, forecast intervals and portfolio evaluation. None of those tests is part of this homework output. The important point is that the classical model is not winning because returns are easy. It is competitive because it is small and tightly specified. When signal is weak, a small model that avoids overfitting can match or beat a flexible model.

The price block looks stronger. ARMA-GARCH reaches price NRMSE 0.0063 in the transformed specification, compared with 0.0113 for the Exercise 2 GP. Figure 7 shows why. A price forecast is built as $\widehat{p}_{t+1} = p_t + \widehat{r}_{t+1}$, so the model inherits the observed level before adding a return forecast. That construction tracks the index level because most of tomorrow's log price is already today's log price. It should not be mistaken for a high Sharpe-ratio return signal. The same dependence on yesterday's price level made the GP look strong in Exercise 2.

The direction block is more informative when MCC is read beside accuracy. Yeo-Johnson ARMA-GARCH matches the majority baseline accuracy at 0.6246, yet it reaches a positive MCC of 0.0627. The plain ARMA-GARCH model has lower accuracy, 0.6000, with MCC 0.0759. That tells us the classical baselines make some balanced sign decisions, while the GP direction classifier mostly reproduces the majority class. Figure 8 is not a trading rule by itself. It checks whether the model switches signs in a way that lines up with down months. The relation is positive, small and sample dependent.

The result has a narrow scope. In this split, where the signal-to-noise ratio is low, a carefully specified classical baseline extracts about as much forecasting content as the more flexible ML models. A final limitation is that all comparisons use one chronological split. The conclusion is convincing for this homework sample. It should not be sold as a universal ranking of model families. This strengthens the main lesson rather than contradicting Exercise 2. The ML models were not beaten by a very rich benchmark. They were matched by a standard ARMA-GARCH model that is built for weak means and persistent volatility. In this sample, that structure is exactly what the data reward.

4 Summary of Lessons

Financial modelling begins from the instruments themselves: bonds, stocks, derivatives and structured products differ in payoff, risk profile and pricing logic, and no quantitative analysis is meaningful without understanding those differences first. The organising principle behind valuation is no-arbitrage, which states that two portfolios with identical future payoffs must carry the same present value. This benchmark also sets the main warning for prediction: if markets are efficient and prices already reflect all available information, forecasting excess returns is inherently difficult, and any claimed signal must survive rigorous out-of-sample testing before it can be trusted.

Log returns are the natural object for time-series modelling because they aggregate additively across periods, which allows linear lag structures to operate directly on the return sequence. In practice, financial returns are far from normally distributed:

they exhibit volatility clustering, heavy tails and occasional extreme observations that a Gaussian assumption would severely underweight. Volatility is better treated as a persistent conditional risk state than as a fixed parameter. The EWMA estimator captures this by giving geometrically decaying weight to past squared returns, so a large shock enters tomorrow's variance estimate and then fades slowly rather than disappearing after one observation. Exercise 1 confirmed this empirically: the EWMA volatility networks contained 21 weekly edges against only 6 in the return networks, showing that the smoothed second moment carries memory across assets long after the signed return has faded.

A useful decomposition separates a financial time series into a deterministic component, a predictable conditional mean and an unpredictable innovation. ARMA models target the mean component while GARCH models address the conditional variance by allowing the innovation size to feed back into next-period risk. This separation proved important in Exercise 3: both ARMA-GARCH specifications produced persistence values of $\hat{\alpha} + \hat{\beta} \approx 0.98$, confirming that nearly all of a monthly variance shock carries into the next period's forecast. Neural networks extend this toolkit by approximating nonlinear functions of the inputs, but their advantage over simpler models depends on whether the target contains enough stable signal to reward the additional flexibility. When the signal-to-noise ratio is low, a tightly specified classical model can match or outperform a flexible one.

Granger causality is a forecasting concept rather than a structural claim: one series Granger-causes another if its past improves linear prediction of the receiver after the receiver's own history is already included. Lag selection and sample size interact in ways that can distort inference, because adding lags improves in-sample fit while consuming degrees of freedom that a short sample cannot spare. Exercise 1 made this risk visible: the monthly return network produced 14 edges against only 6 at the weekly frequency, yet with only 23 monthly observations several of those edges used lags near the maximum allowed horizon and should be read as leads to investigate rather than stable trading delays. Feature selection addresses a related problem on the predictor side: financial candidate variables are often noisy, persistent and mutually correlated, so a disciplined selection step is necessary to avoid fitting historical coincidences rather than stable relationships.

Recurrent neural networks model sequential dependence by carrying a hidden state forward through time, and the LSTM architecture refines this with gates that control how much past information is retained or overwritten at each step. Gaussian processes take a complementary approach: rather than learning a parametric function, they define a prior over functions through a kernel that measures similarity between observations and produce predictions as posterior means weighted by that similarity. Both model families must be evaluated on strictly chronological test sets, because apparent fit can come from persistence or leakage rather than genuine predictive content. Exercise 2 demonstrated this directly: the GP reached 98.87% `pcorrect` on log prices by exploiting the near-unit-root level of the index, yet its direction MCC was 0.000, confirming that a high table score on a persistent target does not imply a usable return signal.

5 Contributions

All authors contributed equally.

References

Argimiro Arratia, Alejandra Cabaña, and José Rafael León. Deep and Wide Neural Networks Covariance Estimation. In *Artificial Neural Networks and Machine Learning - ICANN 2020*. Springer, 2020. doi: 10.1007/978-3-030-61609-0_16. Pages 195-206.

Amit Goyal and Ivo Welch. A Comprehensive Look at the Empirical Performance of Equity Premium Prediction. *The Review of Financial Studies*, 21(4), 2008. Pages 1455-1508.

Alex Tank, Ian Covert, Nick Foti, Ali Shojaie, and Emily B. Fox. Neural Granger Causality. *arXiv preprint arXiv:1802.05842*, 2021.

U.S. Securities and Exchange Commission. Statement on the Approval of Spot Bitcoin Exchange-Traded Products. https://www.sec.gov/newsroom/speeches-statements/gensler-statement-spot-bitcoin-011023, 2024. Accessed May 14, 2026.